

Estruturas de Seleção

Até então, fluxo era contínuo. Agora, alteração do fluxo será:

- Baseado em uma decisão (IF)
- Baseado em um menu (Switch/Case)

Quando usar Estrutura de Seleção IF ?

- Executar, ou não, um comando, baseado em uma condição.

Condições: - podem ser representados por "Expressões booleanas"

Comando(s): - Pode ser Único ou a partir de Bloco de comandos { }

```
if (expressão booleana)  
comando;
```

Semântica

Se a condição for verdadeira, o *comando* será executado; se não for, o fluxo de execução segue após o comando sem executá-lo.

```
if (expressão booleana) {  
comando_1;  
comando_2;  
.....  
}
```

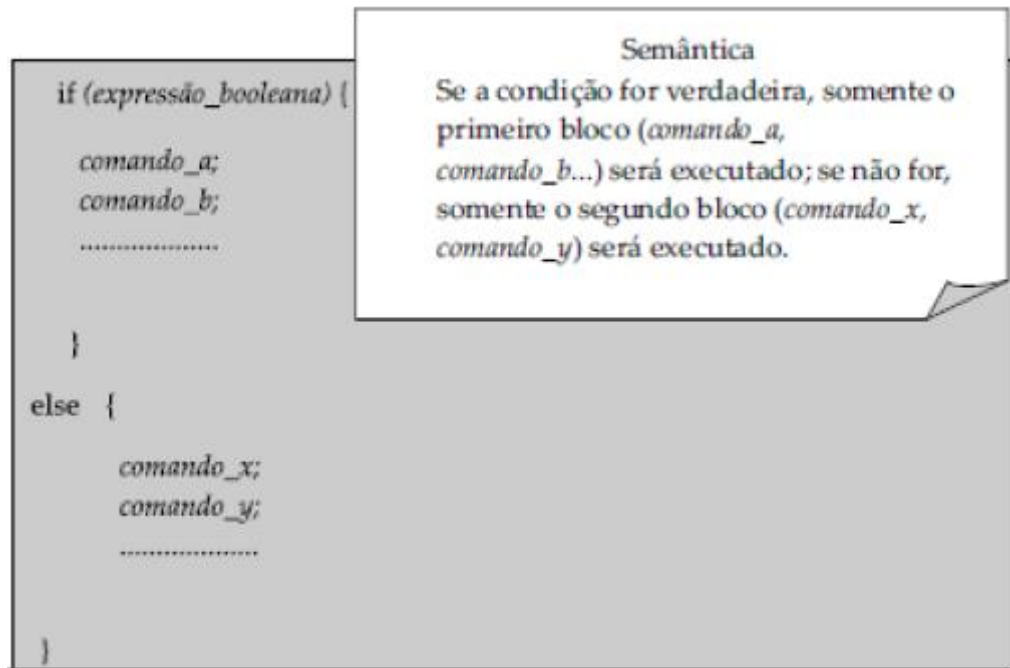
Semântica

Se a condição for verdadeira, o bloco de comandos será executado; se não for, o fluxo de execução segue após o bloco, sem executá-lo.

```
if (expressão booleana)  
comando_a;  
else  
comando_b;
```

Semântica

Se a condição for verdadeira, somente o *comando_a* será executado; se não for, somente o *comando_b* será executado.



Exemplos: Qual as condições para executar os comandos abaixo?

Em português:

se (numero > 0) então
raiz = $\sqrt{\text{numero}}$

Em Java:

```
if (numero > 0)
    raiz = Math.sqrt (numero);
```

Qual as condições para executar os comandos abaixo?

Em português:

se (média $\geq$ 5,0) então
escreva "Aprovado"
senão escreva "Reprovado"

Em Java:

```
if (media >= 5.0)
    System.out.println ("Aprovado");
else
    System.out.println "Reprovado");
```

Mais um exemplo exemplo : (else if)

```
if (media >= 9.0)
    System.out.println ("Otimo");
else if (media >= 8.0)
    System.out.println ("Bom");
else if (media >= 6.0)
    System.out.println ("Aprovado");
else
    System.out.println ("Reprovado");
```

Quando usar Estrutura de Seleção SWITCH /CASE ?

– Selecionar um entre vários comandos.

Condição:

- Comparação de uma variável com um valor exato.
- Byte, char, short ou int.

Comando(s)

- Único comando ou
- Bloco de comandos

```
switch (variável) {
    case valor 1 : comando_a;
                  comando_b;
                  ...
                  break;

    case valor 2 : comando_x;
                  comando_y;
                  ...
                  break;

    ...

    case valor N : comando_m;
                  comando_n;
                  ...
                  break;

    [default    :bloco_de_comandos; ]
}
```

Semântica

O valor da variável é comparado com valor 1. Se igual, executa os comandos que fazem parte deste bloco até atingir o comando break, quando então segue o fluxo de execução após o switch. Se não é igual, a variável é comparada com valor 2, valor 3 e assim sucessivamente. Se o valor da variável não for igual a nenhum dos valores especificados, então executa o bloco de comandos que estão em default.

Exemplo

Em português:

```
Valor da variável (dia)
caso seja 2: escreva ("Segunda-feira")
    saia
caso seja 3: escreva ("Terça-feira")
    saia
caso seja 4: escreva ("Quarta-feira")
    saia
caso seja 5: escreva ("Quinta-feira")
    saia
caso seja 6: escreva ("Sexta-feira")
    saia
nenhum dos acima: escreva ("Fim de semana")
```

Em Java:

```
switch (dia)
case 2: System.out.println ("Segunda-feira");
    break;
case 3: System.out.println ("Terça-feira");
    break;
case 4: System.out.println ("Quarta-feira");
    break;
case 5: System.out.println ("Quinta-feira");
    break;
case 6: System.out.println ("Sexta-feira");
    break;
default: System.out.println ("Fim de semana");
```

Exemplo

```
int i =0;
switch(i){
    case 0 : i++;
    case 1 : i++;
}
System.out.println(i);
```

Exemplo de estruturas de seleção aplicadas dentro de métodos

- Métodos utilizando comandos de seleção

- Método que recebe um número inteiro e exibe na tela a mensagem "Negativo" ou "Não negativo", conforme o caso.

```
public void exibeMensagem(int num){
    if (num < 0)
        System.out.println ("Negativo");
    else
        System.out.println ("Não negativo");
}
```

- Método que recebe um número inteiro e retorna a mensagem "Par" ou "Impar", conforme o caso.

```
public String verificaParOuImpar(int num){  
    if (num%2==0)  
        return "Par" ;  
    else  
        return "Impar"; //aqui else é opcional  
}
```

- Método que recebe dois números reais e retorna o maior. Se iguais, retorna qualquer um.

```
public double maiorDosDois(double a, double b){  
    if (a>b)  
        return a;  
    else  
        return b; //aqui else é opcional  
}
```

- Método que recebe dois números inteiros e retorna uma das seguintes mensagens, conforme o caso: "Os dois são diferentes", ou "Os dois são iguais".

```
public String comparaNumeros(int umNum, int outroNum){  
    if (umNum != outroNum)  
        return ("Os dois são diferentes");  
    else  
        return ("Os dois são iguais");  
}
```

- Método que recebe dois nomes de pessoas, do tipo String, e retorna true se os dois nomes são iguais e false se são diferentes. Observe que a comparação de strings é realizada com os métodos da classe String, pois String é um objeto.

```
public boolean iguais(String umNome, String outroNome){  
    if (umNome.equalsIgnoreCase (outroNome))  
        return true;  
    else //aqui else é opcional  
        return false;  
}
```

- Método que recebe a idade de uma pessoa, e classifica-a da seguinte forma: até 12 anos, "Infantil"; de 13 a 18, "Adolescente"; de 19 a 24, "Jovem"; de 25 a 70, "Adulto"; mais de 70 anos, "Terceira idade".

```
public String faixaEtaria (int idade){  
    String msg = "Terceira idade";  
    if (idade<= 12 )  
        msg = "Infantil";  
    else if (idade<= 18)  
        msg = "Adolescente";  
    else if (idade< 25 )  
        msg = "Jovem";  
    else if (idade<= 70 )  
        msg = "Adulto";  
    return msg;  
}
```

- Método que recebe o ano e retorna true ou false, indicando se o ano é bissexto ou não.

```
public boolean ehBissexto(int ano) {  
    if ((ano % 4 == 0)  
        &&(ano % 100 != 0)  
        || (ano % 400 == 0))  
        return true;  
    else return false; //aqui else é opcional  
}
```

